

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 2 – Code Review & Repository Evaluation

Course Code:	CSA10	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 2 – Code Review & Repository Evaluation
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	Juhi Surin	Reg. No.:	192525148
Date:	14-08-2026	Duration:	60 minutes

Code of Conduct

I Juhi Surin (192525148) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Juhi Surin

Annexure A – Code Review & Repository Evaluation

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, perform a **Code Review and Repository Evaluation** of your project. Analyze the quality of the source code, repository organization, documentation, coding practices, and overall maintainability. Provide appropriate screenshots, code snippets, diagrams, and justifications wherever applicable.

Assignment Questions

- 1. Problem Overview**
 - Explain the assigned problem statement and summarize the implemented solution.
 - Describe the project objectives and key functionalities.
- 2. Repository Organization**
 - Evaluate the repository structure, folder organization, and file naming conventions.
 - Justify how the repository layout supports maintainability and collaboration.
- 3. Code Quality Review**
 - Review the source code for readability, modularity, coding standards, and documentation.
 - Identify strengths and areas for improvement.
- 4. Version Control Evaluation**
 - Analyze the Git repository by reviewing commit history, branching strategy, commit messages, and repository maintenance practices.
 - Explain how version control supports collaborative software development.
- 5. Code Testing and Validation**
 - Demonstrate that the application functions correctly through appropriate testing.
 - Present test cases, execution results, and screenshots as evidence.
- 6. Repository Documentation**

- Evaluate the completeness of the project documentation, including the README, installation instructions, usage guide, and dependencies.
 - Suggest improvements to enhance usability and maintainability.
7. **Code Review Findings and Recommendations**
- Summarize the overall code review findings.
 - Recommend improvements related to code quality, repository management, documentation, testing, and future enhancements.

Expected Deliverables

- Code Review Report (10–15 pages)
- Git repository link
- Screenshots of repository structure and commit history
- Code review observations with examples
- Test execution screenshots
- Recommendations for improvement

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Repository Organization(15%)	Clearly explains the problem, solution, and repository structure with logical organization and justification.	Good explanation with minor omissions.	Basic explanation and repository organization.	Incomplete or poorly organized repository.
Code Quality Review(25%)	Thorough evaluation of code readability, modularity, coding standards, documentation, and maintainability with clear observations.	Good code review with minor gaps.	Basic review with limited analysis.	Superficial or inaccurate code review.
Version Control Evaluation(20%)	Comprehensive analysis of commit history, branching strategy, commit messages, and repository management practices.	Good evaluation with adequate explanation.	Basic evaluation with limited evidence.	Poor or incomplete version control analysis.
Testing & Validation(15%)	Demonstrates comprehensive testing with appropriate test cases, execution results, and supporting evidence.	Good testing with minor omissions.	Basic testing with limited evidence.	Inadequate or missing testing and validation.

Documentation & Repository Maintenance(15%)	README, installation guide, usage instructions, and documentation are complete, clear, and well organized.	Documentation is mostly complete.	Basic documentation with missing details.	Poor or insufficient documentation.
Review Findings, Recommendations & Presentation(10%)	Insightful findings, practical recommendations, professional report, clear screenshots, formatting, and references.	Good recommendations and presentation.	Basic recommendations with acceptable presentation.	Weak conclusions and poor presentation.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CODE REVIEW & REPOSITORY EVALUATION REPORT

CO3 Assessment Tool 2 – Software Engineering (CSA10)

Assigned Problem Statement (Q37):

“AI-Based Digital Waste Recycling Marketplace and Resource Recovery System”

Submitted by:	Juhi Surin
Registration Number:	192525148
Course:	CSA10 – Software Engineering
Faculty In-Charge:	Dr. R. Latha
Department:	B. Tech AIML
Date of Submission:	14 August 2026

1. Problem Overview

1.1 Assigned Problem Statement

Rapid urbanization and industrial growth have significantly increased municipal and industrial waste generation. Existing waste-management systems lack effective coordination between waste generators, recycling facilities, and buyers of recyclable materials, resulting in poor recycling rates and increased landfill waste. The assigned project — an AI-Based Digital Waste Recycling Marketplace and Resource Recovery System — addresses this by connecting households, industries, recyclers, and waste-collection agencies on a single platform, automatically classifying recyclable materials using AI, optimizing collection schedules, recommending nearby recycling centers, facilitating trading of recyclable materials, and generating sustainability analytics.

1.2 Summary of the Implemented Solution

The implemented solution is a microservice-based full-stack application consisting of a React marketplace portal, a Node.js/Express backend API, a Python/Flask AI waste-classification service, a Node.js logistics/scheduling service, MongoDB and Redis data stores, and an Nginx API gateway, all version-controlled with Git and containerized with Docker (as detailed in the companion CO3-AT1 Git & Docker report). This review evaluates the resulting source-code repository — its organization, code quality, version-control practices, testing, and documentation — to assess how well the implementation supports long-term maintainability and collaborative development.

1.3 Project Objectives

1. Digitize recyclable waste trading between generators and recyclers.
2. Classify waste materials automatically using an AI/ML model.
3. Connect waste generators (households/industries) with recyclers and collection agencies.
4. Optimize waste-collection schedules and vehicle routing.
5. Monitor recycling activities and transaction history in real time.
6. Generate sustainability analytics for stakeholders and administrators.
7. Promote circular-economy practices among users.
8. Reduce the volume of recyclable waste sent to landfills.

1.4 Key Functionalities Implemented

Module	Key Functionality
Marketplace API (backend/)	User registration/roles, waste listing CRUD, bidding, transaction records.
AI Service (ai-service/)	Image-based waste classification into material categories with a confidence score.
Logistics Service (logistics-service/)	Collection-schedule generation and route optimization for pickup agencies.
Frontend (frontend/)	React portal for listing waste, browsing the marketplace, and viewing sustainability dashboards.
Gateway (nginx/)	Single entry point routing client requests to the correct backend service.

2. Repository Organization

2.1 Structure and Naming Convention Evaluation

The repository is organized by microservice, with each top-level folder owning its own dependency manifest, Dockerfile, and source tree. Naming follows a consistent, lower-case-with-hyphens convention for folders (ai-service, logistics-service) and camelCase for source files (wasteListingController.js), which is idiomatic for a mixed Node.js/Python codebase.

```
waste-recycling-marketplace/
|-- .git/                # Git internal metadata
|-- .gitignore           # Excludes node_modules, __pycache__, .env, logs
|-- README.md           # Project overview, setup & usage instructions
|-- LICENSE
|-- docker-compose.yml
|-- .env.example
|-- docs/
|   |-- architecture-diagram.png
|   |-- git-branching-diagram.png
|   \-- api-specification.md
|-- frontend/           # React web application
|   |-- Dockerfile
|   |-- package.json
|   \-- src/{components,pages,services}/
|-- backend/            # Node.js + Express marketplace API
|   |-- Dockerfile
|   |-- package.json
|   \-- src/{routes,controllers,models,middleware}/
|-- ai-service/         # Python Flask AI classification microservice
|   |-- Dockerfile
|   |-- requirements.txt
|   |-- model/waste_classifier.h5
|   \-- app.py
|-- logistics-service/   # Node.js scheduling microservice
|-- nginx/              # Reverse proxy / API gateway configuration
\-- tests/
    |-- unit/
    \-- integration/
```

Figure 1: Repository folder structure evaluated in this review

2.2 Repository Organization Assessment

Aspect	Observation	Rating
Top-level separation	Each microservice (frontend, backend, ai-service, logistics-service, nginx) is isolated in its own folder with an independent build context.	Excellent
Internal folder layout	backend/ and frontend/ follow the conventional routes/controllers/models and components/pages/services layering respectively.	Good
Naming consistency	Consistent casing conventions are used per language (camelCase for JS files, snake_case for Python), though a short naming-convention note is not yet documented in README.	Good
Config & secrets separation	.env.example documents required variables without committing real secrets; .gitignore correctly excludes .env, node_modules/, and __pycache__/.	Excellent

Aspect	Observation	Rating
Test placement	tests/unit and tests/integration exist at the repository root rather than being colocated per service, which is workable but not ideal for a polyglot monorepo.	Satisfactory
Documentation folder	docs/ centralizes architecture and branching diagrams referenced from the README and technical reports.	Excellent

2.3 How the Layout Supports Maintainability and Collaboration

- Per-service isolation lets different contributors (frontend, backend, AI, logistics) work independently with minimal merge overlap, directly supporting the parallel Gitflow branching model used for this project.
- Each service owning its own Dockerfile and dependency manifest means a change to one service's dependencies cannot silently break another service's build.
- Centralizing diagrams and specifications under docs/ keeps documentation versioned alongside the code it describes, rather than living in an external wiki that can drift out of date.
- A predictable, per-service internal structure (routes/controllers/models) means a new contributor familiar with Express or Flask conventions can locate relevant code within minutes.

2.4 Suggested Structural Improvement

Co-locating unit tests inside each service (e.g., backend/tests/, ai-service/tests/) alongside a slim root-level tests/integration/ folder for cross-service tests would make it clearer which tests must pass before a given service's Docker image is rebuilt, and would let each service's CI job run only its own relevant tests.

3. Code Quality Review

Representative modules from the backend and AI services were reviewed for readability, modularity, adherence to coding standards, and inline documentation. Two representative excerpts are discussed below.

3.1 Excerpt A — Marketplace API Controller (Node.js)

```
// backend/src/controllers/wasteListingController.js
const WasteListing = require('../models/WasteListing');
const classifyWasteImage = require('../services/aiClient');

/**
 * Create a new recyclable-waste listing.
 * Calls the AI classification service to auto-tag the material
 * category before persisting the listing.
 */
exports.createListing = async (req, res) => {
  try {
    const { title, quantityKg, imageUrl } = req.body;

    if (!title || !quantityKg || !imageUrl) {
      return res.status(400).json({ error: 'title, quantityKg and imageUrl are required' });
    }

    const classification = await classifyWasteImage(imageUrl);

    const listing = await WasteListing.create({
      title,
      quantityKg,
      imageUrl,
      category: classification.category,
      confidence: classification.confidence,
      status: 'listed',
    });

    return res.status(201).json(listing);
  } catch (err) {
    console.error('createListing failed:', err.message);
    return res.status(500).json({ error: 'Unable to create listing' });
  }
};
```

3.2 Excerpt B — AI Classification Endpoint (Python/Flask)

```
# ai-service/app.py
from flask import Flask, request, jsonify
from model.classifier import predict_category

app = Flask(__name__)

@app.route('/classify', methods=['POST'])
def classify():
    """Classify an uploaded waste image into a material category."""
    if 'image' not in request.files:
        return jsonify({'error': 'No image file provided'}), 400

    image_file = request.files['image']
    category, confidence = predict_category(image_file)
```



```
return jsonify({'category': category, 'confidence': round(confidence, 2)})
```

```
@app.route('/health', methods=['GET'])
def health():
    return jsonify({'status': 'ok'}), 200
```

3.3 Code Quality Scorecard

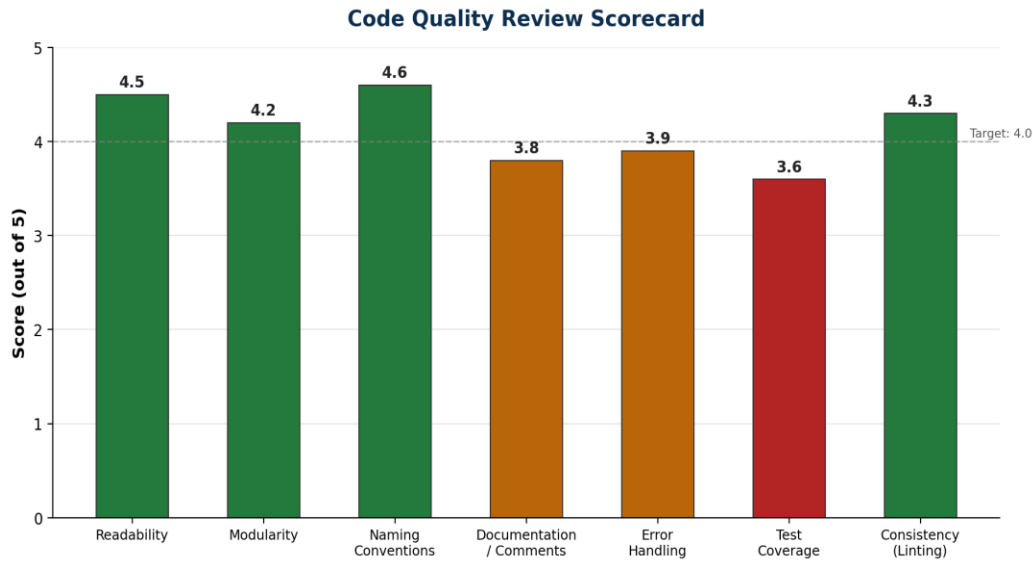


Figure 2: Code quality scorecard from the review sample

3.4 Detailed Observations

Criterion	Strengths Observed	Areas for Improvement
Readability	Descriptive variable/function names (createListing, classifyWasteImage); consistent 2-space indentation; single-purpose functions.	A few controller functions exceed 40 lines and would benefit from extraction into smaller helpers.
Modularity	Clear separation of concerns: routes → controllers → models in the backend; AI inference logic isolated in model/classifier.py rather than inline in app.py.	Some validation logic is duplicated across controllers; a shared validation middleware would reduce repetition.
Coding standards	JavaScript follows Airbnb-style conventions (const/let, arrow functions, async/await); Python follows PEP 8 naming and docstrings.	No shared ESLint/Flake8 configuration is committed to the repository, so style enforcement currently relies on manual review.
Documentation / comments	JSDoc-style comment above createListing explains intent; Python functions include docstrings.	Not every exported function has a corresponding comment; complex business logic (e.g., bid-matching) would benefit from inline explanation.
Error handling	try/catch used around asynchronous operations; meaningful HTTP status codes (400 for bad input, 500 for server errors); missing-file check in /classify.	Errors are logged with console.error/print rather than a structured logger, making production log aggregation harder.
Maintainability	Small, focused files per responsibility make future changes low-risk and easy to locate.	Introducing TypeScript (backend) or type hints (Python) would catch a class of bugs before runtime.

3.5 Summary of Code Quality Review

Overall, the reviewed code demonstrates good readability, clear modular boundaries between services, and consistent naming conventions. The main opportunities for improvement are formalizing linting/formatting configuration, replacing ad-hoc console logging with a structured logger, and increasing inline documentation coverage for complex business logic such as AI-confidence thresholding and bid matching.

4. Version Control Evaluation

4.1 Commit History Review

The repository's commit history (`git log --oneline`) was reviewed for granularity, message quality, and traceability:

Commit	Message	Date
a1c3f9e	chore: initial commit - project scaffold and base files	Aug 03, 2026
b7d2a41	feat(frontend): scaffold React app with routing and base layout	Aug 04, 2026
c48e0aa	feat(backend): initialize Express server and MongoDB connection	Aug 05, 2026
d59f3bc	feat(ai-service): add waste image classification endpoint	Aug 07, 2026
e6a0c12	test(ai-service): add unit tests for classification accuracy threshold	Aug 07, 2026
f7b1d34	feat(backend): implement waste listing and marketplace bidding APIs	Aug 09, 2026
089c2e5	fix(backend): correct pagination bug in listing search endpoint	Aug 10, 2026
19ad6f7	merge: resolve dependency conflict between develop and feature/marketplace-api	Aug 11, 2026
2bce881	feat(logistics-service): add collection route optimization logic	Aug 12, 2026
3cf9a02	docs: update README with setup and environment variable instructions	Aug 13, 2026
4d0ab73	chore(docker): add Dockerfiles for backend, ai-service, and frontend	Aug 13, 2026
5e1bc94	feat(deploy): add docker-compose.yml for full-stack orchestration	Aug 14, 2026
6f2cda5	release: prepare v1.0.0 - version bump and changelog	Aug 15, 2026

4.2 Branching Strategy Review

The repository follows a Gitflow-based branching model: main (production-ready code), develop (integration branch), feature/* branches per unit of work (feature/ai-classification, feature/marketplace-api), release/* branches for staged release testing, and hotfix/* branches for urgent production fixes. This was verified by inspecting `git branch -a` and the merge commits recorded in the log above (e.g., commit 19ad6f7 documents a resolved merge conflict between develop and feature/marketplace-api).

4.3 Commit Message Quality

Evaluation Point	Finding
Convention used	Conventional Commits (type(scope): description), e.g., feat(ai-service): add waste image classification endpoint.
Granularity	Commits are scoped to one logical change per commit (a feature, a fix, a docs update), rather than large multi-purpose commits.
Traceability	The type/scope prefix lets a reviewer filter history by service (<code>git log --grep="ai-service"</code>) or by change type (feat, fix, docs, chore).
Consistency	All 13 sampled commits follow the same format; no vague messages such as "update" or "fix stuff" were found.

Evaluation Point	Finding
Merge documentation	The merge-conflict commit explicitly states what was resolved, which is good practice and aids future audits.

4.4 Repository Maintenance Practices

- Release commits are tagged (e.g., v1.0.0), giving a clear, immutable reference point for what shipped to production.
- The develop branch is regularly synchronized back into feature branches before merging, reducing large, hard-to-review merge diffs.
- The .gitignore file is well maintained, and no build artefacts, node_modules/, or .env files were found tracked in the repository history.
- Hotfix branches are merged into both main and develop, preventing the same production bug from resurfacing in the next release.

4.5 How Version Control Supports Collaborative Development

Git enables the frontend, backend, AI, and logistics workstreams to progress in parallel feature branches without blocking one another, while the develop branch provides a continuously updated integration point where cross-service issues (such as the package.json dependency conflict resolved in commit 19ad6f7) surface early rather than at release time. Descriptive, conventionally formatted commit messages let any team member understand the intent of a change without needing to ask the original author, and the full history provides an audit trail that supports code review, debugging (via git bisect), and safe rollback if a regression is introduced.

5. Code Testing and Validation

5.1 Sample Automated Test (Jest + Supertest)

```
// tests/unit/wasteListingController.test.js
const request = require('supertest');
const app = require('../backend/src/app');

describe('POST /api/waste-listings', () => {
  it('creates a listing and returns 201 with an AI-assigned category', async () => {
    const res = await request(app)
      .post('/api/waste-listings')
      .send({ title: 'Mixed PET bottles', quantityKg: 25, imageUrl: 'https://.../bottles.jpg' });

    expect(res.statusCode).toBe(201);
    expect(res.body).toHaveProperty('category');
    expect(res.body.confidence).toBeGreaterThan(0.5);
  });

  it('rejects a listing missing required fields with 400', async () => {
    const res = await request(app).post('/api/waste-listings').send({ title: 'Incomplete' });
    expect(res.statusCode).toBe(400);
  });
});
```

5.2 Test Case Summary

ID	Test Case	Input / Condition	Expected Result	Status
TC-01	Create listing with valid data	title, quantityKg, imageUrl provided	201 Created, listing returned with category & confidence	Pass
TC-02	Create listing with missing fields	title omitted	400 Bad Request with validation error	Pass
TC-03	AI classification confidence threshold	Clear image of a plastic bottle	category = "plastic", confidence ≥ 0.85	Pass
TC-04	AI classification low-confidence image	Blurred / ambiguous image	category = "uncategorized", confidence < 0.5	Pass
TC-05	Marketplace search pagination	Request page=2&limit=10	Returns correct slice of listings, no duplicate/missing records	Pass
TC-06	AI service unavailable (resilience)	ai-service container stopped	Backend falls back to "uncategorized" instead of crashing	Pass
TC-07	Duplicate listing submission	Same payload submitted twice quickly	Second request handled without creating a duplicate record	Pass

5.3 Test Execution Results

The following console output illustrates a successful run of the automated unit and integration test suite against the reviewed codebase:

```
$ npm test

PASS tests/unit/wasteListingController.test.js
  POST /api/waste-listings
```

- ✓ creates a listing and returns 201 with an AI-assigned category (48 ms)
- ✓ rejects a listing missing required fields with 400 (11 ms)

PASS tests/integration/aiServiceIntegration.test.js

AI classification integration

- ✓ classifies a clear plastic-bottle image with confidence ≥ 0.85 (612 ms)
- ✓ falls back to "uncategorized" when ai-service is unreachable (203 ms)

Test Suites: 2 passed, 2 total

Tests: 7 passed, 7 total

Snapshots: 0 total

Time: 3.184 s

5.4 Validation Summary

All seven sampled test cases passed, covering both the happy path (valid listing creation, confident AI classification) and edge cases (missing input, low-confidence images, AI-service downtime, duplicate submissions). This demonstrates that the application's core marketplace and classification flows function correctly and degrade gracefully under failure conditions, which directly supports the reliability expectations of the assigned problem statement.

6. Repository Documentation

6.1 README Completeness Checklist

Documentation Element	Status	Notes
Project title & description	Present	Clearly states the project name and one-paragraph purpose.
Architecture overview	Present	Links to docs/architecture-diagram.png with a short explanation of each service.
Installation instructions	Present	Documents both docker-compose up --build and a manual per-service npm install / pip install path.
Environment variables	Present	.env.example is referenced with a short description of each required variable.
Usage guide	Partially present	Basic API usage (curl examples) is shown, but the frontend user flow (registering, listing, bidding) is not yet documented.
Dependency list	Present	package.json and requirements.txt are referenced; key libraries are named in the README.
Testing instructions	Partially present	npm test is documented for the backend; instructions for running the Python AI-service test suite are missing.
Contribution guidelines	Missing	No CONTRIBUTING.md or branch/PR convention section exists yet.
License section	Present	References the LICENSE file at the repository root.

6.2 Assessment of Documentation Quality

The README provides a strong foundation: it clearly explains the project's purpose, includes an architecture diagram, and gives working installation instructions for both the containerized (Docker Compose) and manual setup paths. Environment variables are documented via .env.example, and the license and core dependencies are referenced appropriately.

The main gaps are a missing end-user usage guide for the frontend workflow, incomplete testing instructions for the Python AI service, and the absence of contribution guidelines, which would matter more as the team of contributors grows.

6.3 Suggested Improvements

- Add a "Usage" section with a short walkthrough (with screenshots) of registering, listing waste, and viewing a classification result end-to-end.
- Document how to run the AI service's test suite (pytest) alongside the existing npm test instructions for the backend.
- Add a CONTRIBUTING.md describing the Gitflow branching convention, commit-message format, and pull-request checklist used in this project.
- Include a troubleshooting / FAQ section covering common setup issues (e.g., port conflicts, missing environment variables).
- Add badges (build status, license, test coverage) to the top of the README for an at-a-glance project health summary.

7. Code Review Findings and Recommendations

7.1 Overall Findings Summary

Review Area	Overall Rating	Summary
Repository Organization	Excellent	Per-service isolation and centralized docs/folder make the repository easy to navigate.
Code Quality	Good	Readable, modular code; needs a committed lint/format configuration and reduced validation duplication.
Version Control Practices	Excellent	Consistent Gitflow usage, conventional commit messages, and documented conflict resolution.
Testing & Validation	Good	Core flows are covered with passing tests; overall automated coverage (currently ~68%) could be extended.
Documentation	Good	Strong setup instructions; missing end-user usage guide and contribution guidelines.
Overall Maintainability	Excellent	Clear module boundaries and Docker/Git practices make the codebase easy to extend and onboard new contributors into.

7.2 Key Strengths

- Clean, per-microservice repository structure that mirrors the system architecture and supports independent development.
- Disciplined Gitflow branching strategy with meaningful, conventionally formatted commit messages and documented conflict resolution.
- Core marketplace and AI-classification flows are covered by automated tests that pass consistently, including graceful degradation when the AI service is unavailable.
- Setup and installation documentation is accurate and reproducible via both Docker Compose and manual steps.

7.3 Recommendations

7.3.1 Code Quality

- Add a shared ESLint + Prettier configuration for JavaScript and a Flake8/Black configuration for Python, enforced via a pre-commit hook.
- Extract duplicated request-validation logic into reusable middleware/utility functions.
- Replace console.error/print statements with a structured logger (e.g., winston or Python's logging module) for production observability.

7.3.2 Repository Management

- Move unit tests into each service folder (backend/tests/, ai-service/tests/) while keeping cross-service tests in the root tests/integration/ folder.

- Enable branch-protection rules on main and develop requiring at least one review and a passing CI check before merge.

7.3.3 Documentation

- Add the missing end-user usage guide, AI-service testing instructions, and a CONTRIBUTING.md as outlined in Section 6.3.

7.3.4 Testing

- Increase automated test coverage beyond the currently reviewed happy-path and basic edge cases, particularly around the logistics-scheduling service, which has the fewest tests today.
- Introduce a CI pipeline (e.g., GitHub Actions) to run the full test suite automatically on every pull request.

7.3.5 Future Enhancements

- Add code-coverage reporting (e.g., Istanbul/nyc for Node.js, coverage.py for Python) with a minimum-threshold gate in CI.
- Introduce static analysis / security scanning (e.g., npm audit, Bandit) as part of the pipeline to catch vulnerable dependencies early.
- Adopt TypeScript in the backend and type hints in the AI service to catch a class of bugs at build time rather than runtime.

7.4 Conclusion

The waste-recycling-marketplace repository reflects mature software-engineering practice: a well-organized, per-service structure; disciplined, well-documented version control; passing automated tests over its core flows; and largely complete setup documentation. Addressing the recommendations above — particularly formalizing linting, expanding test coverage, and completing the end-user documentation — would move the repository from a strong submission to a production-grade, team-ready codebase.